

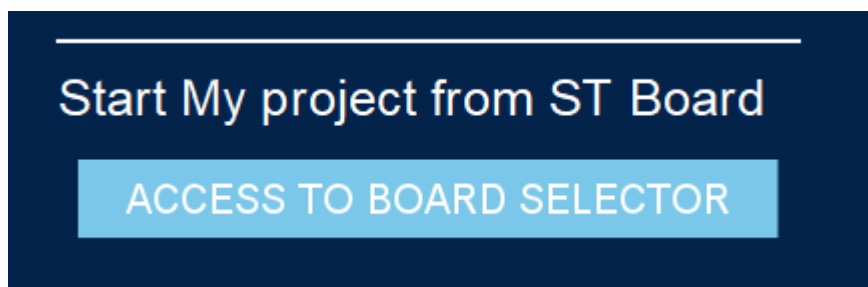
Configuração do Projeto para Atividade 4.

Alunos

- CARLSON VASCONCELOS
- LETICIA FONTENELLE
- VINÍCIUS RIBEIRO

Repo: <https://github.com/Vjfrib/proj-4-embarcados/>

Configuração STMCubeMx NUCLEO (L476RG)



MCU/MPU Selector Board Selector Example Selector Cross Selector

Board Filters

Commercial Part Number: NUCLEO-L476RG

PRODUCT INFO

- Type
- Supplier
- MCU / MPU Series
- Marketing Status
- Price

MEMORY

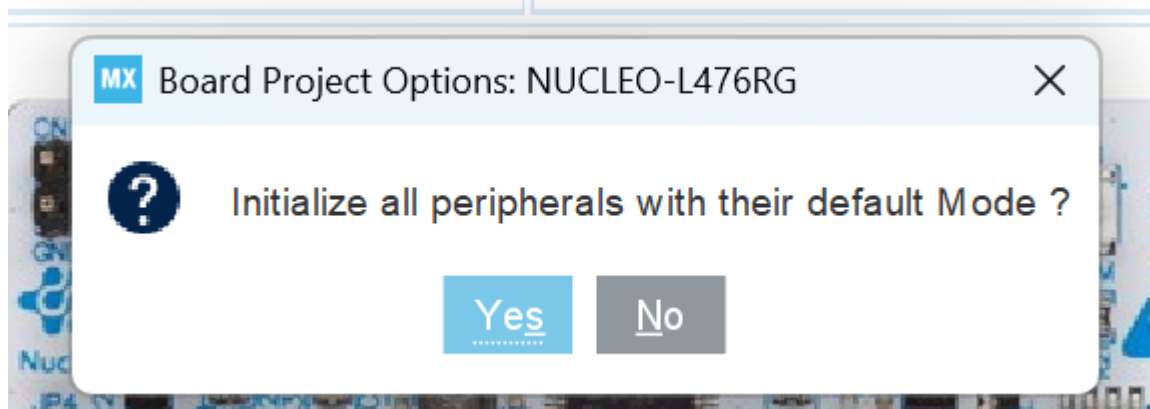
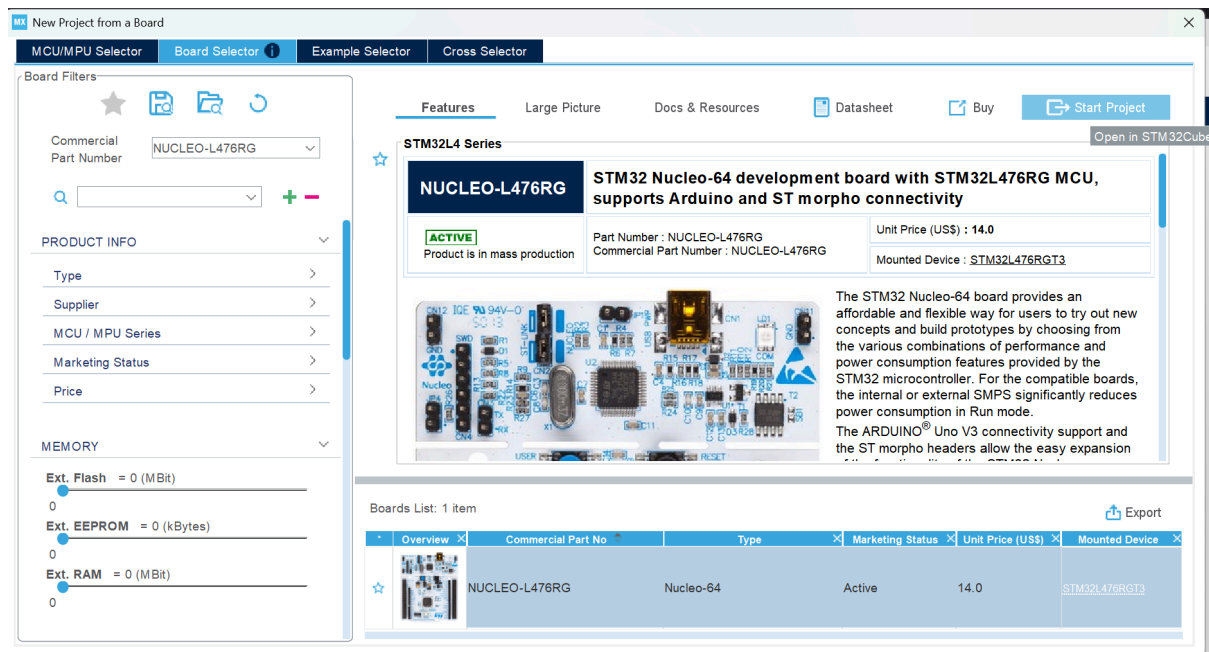
- Ext. Flash = 0 (MBit)
- Ext. EEPROM = 0 (kBytes)
- Ext. RAM = 0 (MBit)

Features Large Picture Docs & Resources Datasheet Buy Start Project

STM32U3B5/U3C5:
bringing high-performance DSP & edge AI to ultralow power designs

Boards List: 1 item

Overview	Commercial Part No	Type	Marketing Status	Unit Price (US\$)	Mounted Device
	NUCLEO-L476RG	Nucleo-64	Active	14.0	STM32L476RGT3



daí, configuramos a conectividade

USART3, para serial com pc:

OBSERVAÇÃO: A conexão USB-MINI da Placa núcleo realiza somente USART2!

7.10 USART communication

The USART2 interface available on PA2 and PA3 of the STM32 microcontroller can be connected to ST-LINK MCU, ST morpho connector, or ARDUINO® connector. The choice can be changed by setting the related solder bridges. By default, the USART2 communication between the target STM32 and ST-LINK MCU is enabled, to support virtual COM port (SB13 and SB14 ON, SB62 and SB63 OFF). If the communication between the

Configuração Serial TTL - USB

para conexão com usart3, utilizamos USB/TTL CH340

datasheet: <https://d26lpennugtm8s.cloudfront.net/stores/198/075/rte/CH340.pdf>

drivers para windows: <https://sparks.gogo.co.nz/ch340.html>

jumper em 3v3-VCC

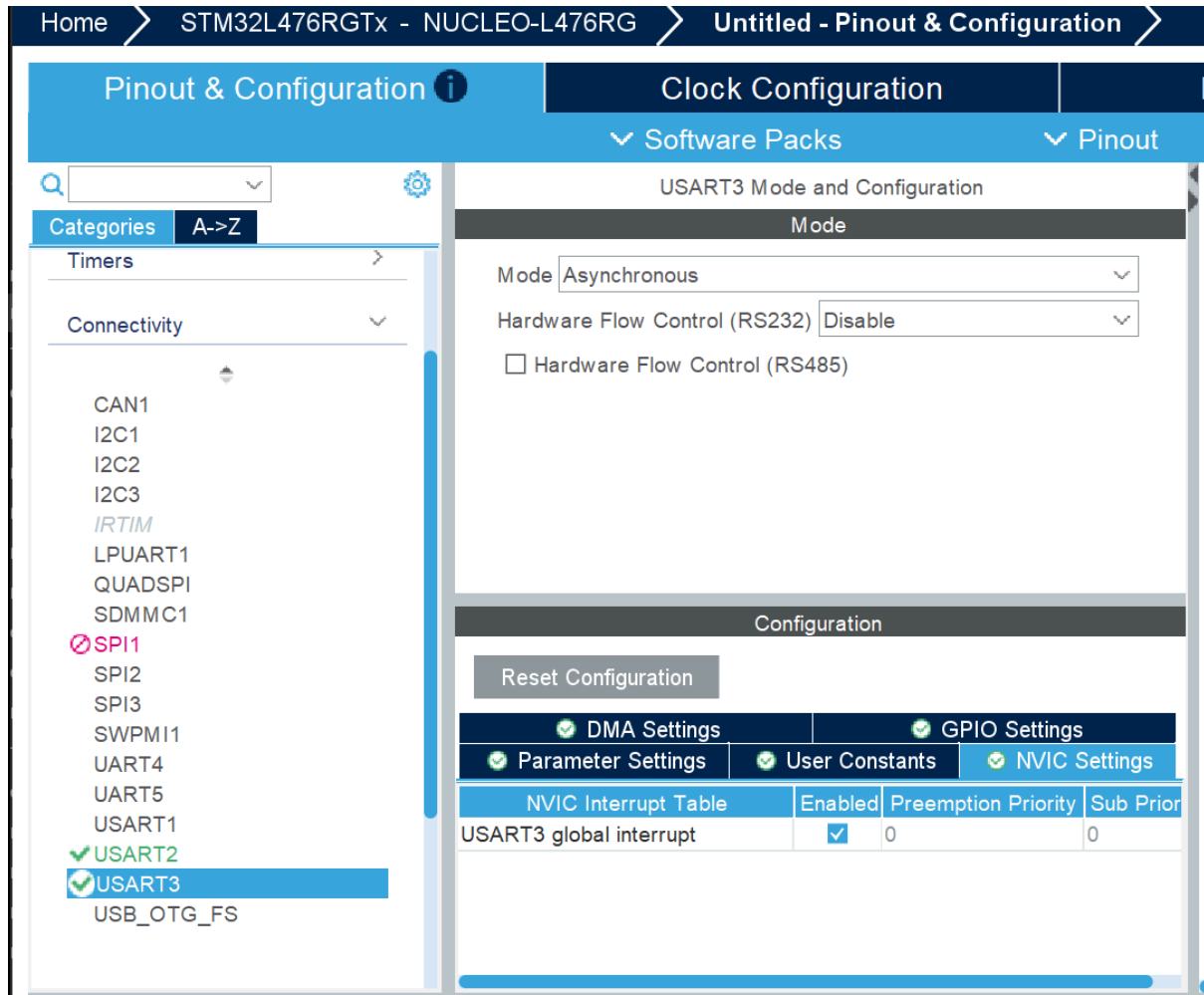
ligações:

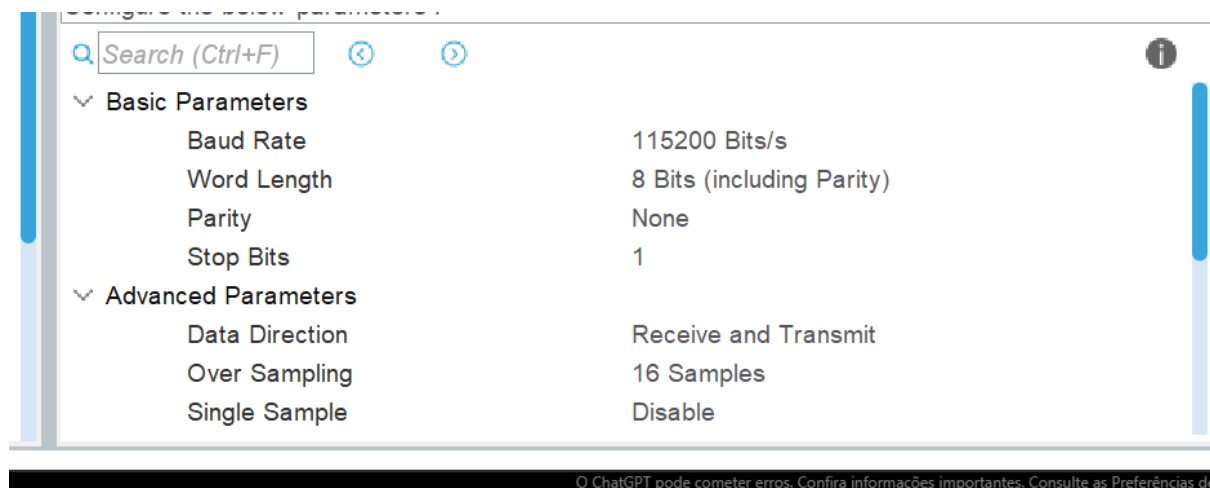
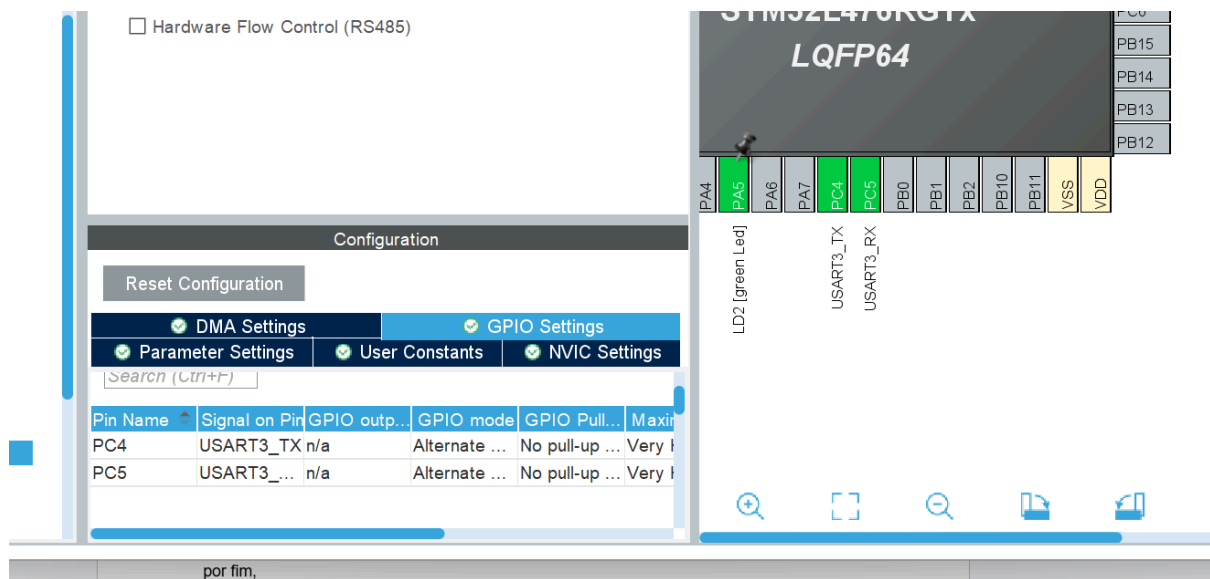
TTL x Nucleo

CH340 GND --- Jumper cinza MxF ---> GND da NUCLEO

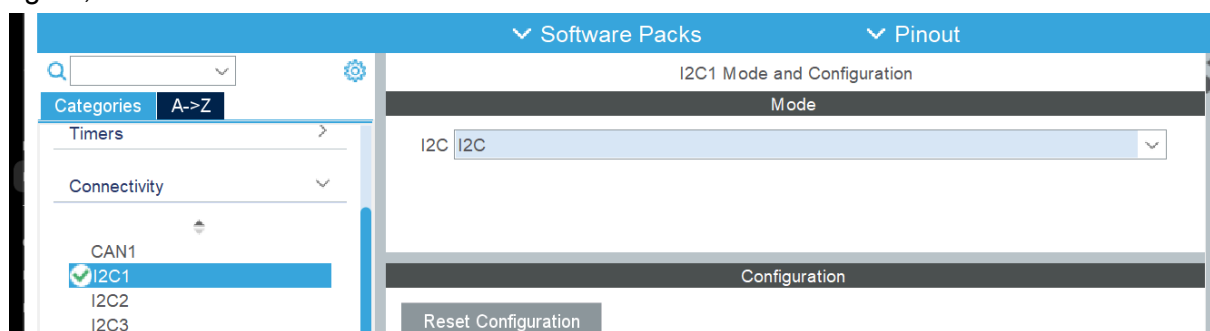
CH340 TXD --- Jumper azul FxF ---> PC5 / USART3_RX (superior-direito)

CH340 RXD --- Jumper roxo FxF ---> PC4 / USART3_TX (inferior direito)

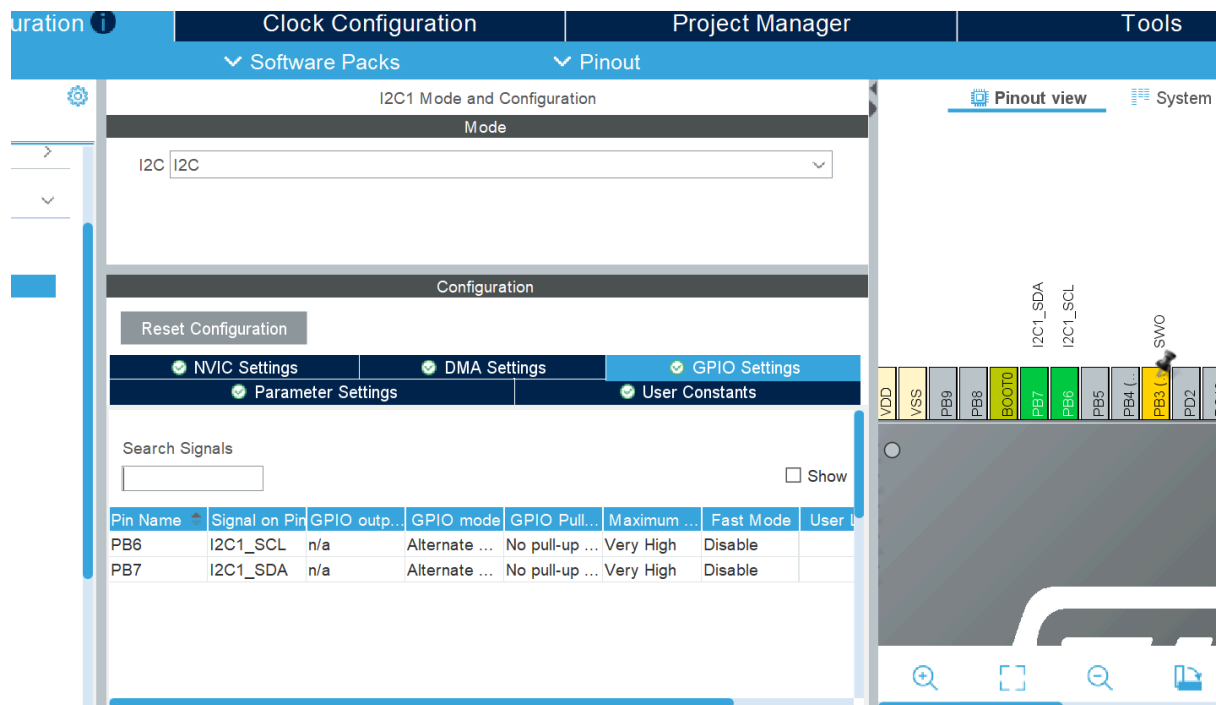




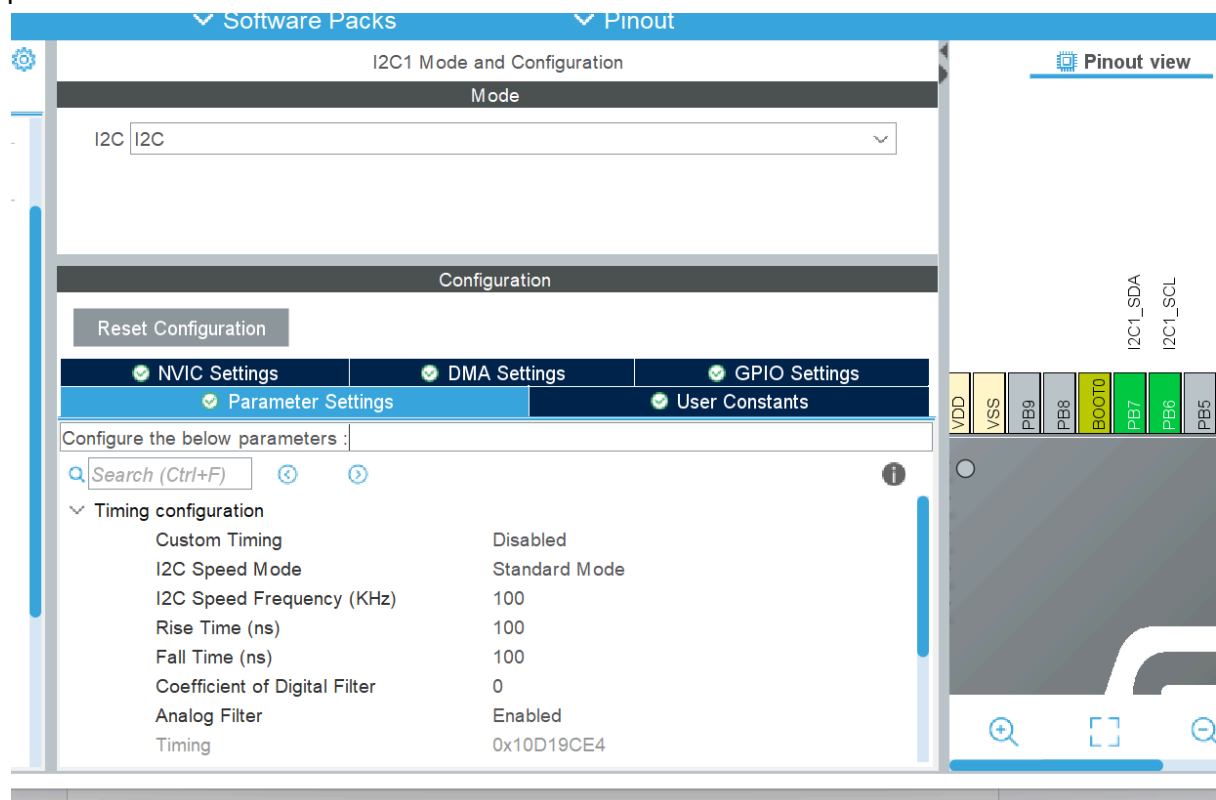
Agora, I2C 1



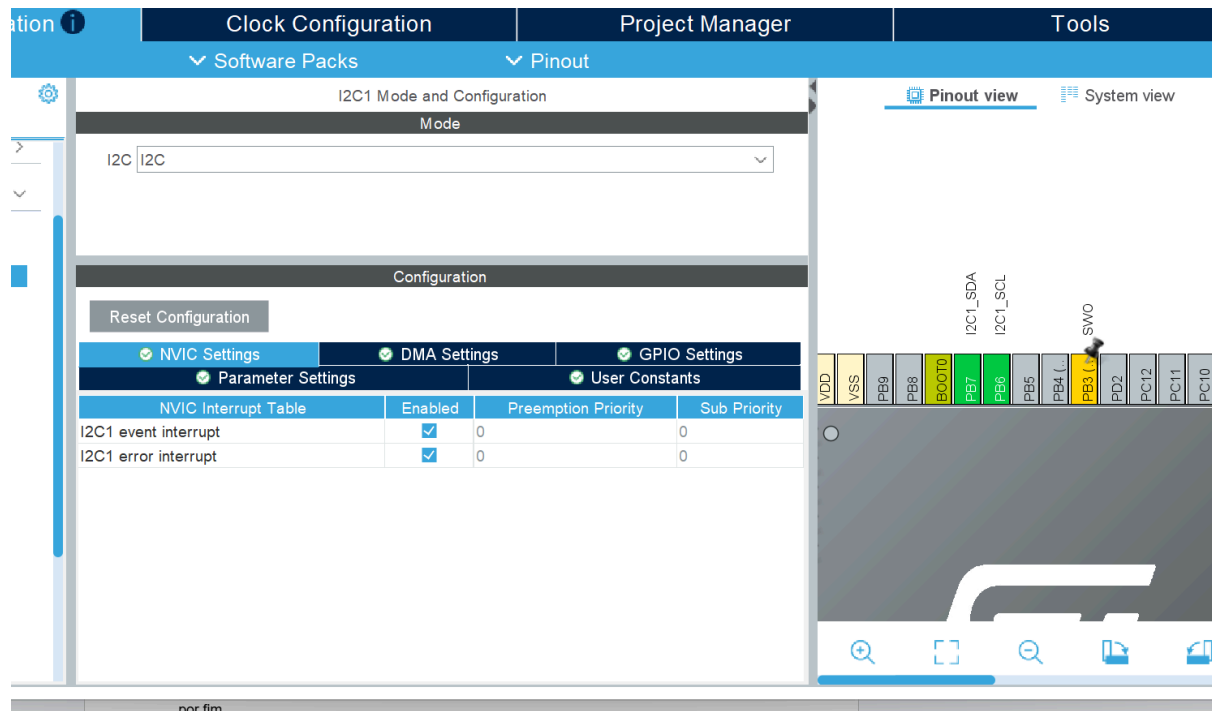
pinos:



parametros



nvic



conectividade fisica:

Na NUCLEO-L476RG, use:

PB8 -> I2C1_SCL

PB9 -> I2C1_SDA

Na placa, isso corresponde aos pinos Arduino:

D15 -> SCL

D14 -> SDA

A placa PCF8591 usa exatamente as linhas SCL, SDA, GND e VCC para comunicação I2C.

No CubeMX, confira se ficou:

PB8 = I2C1_SCL

PB9 = I2C1_SDA

PCF8591	NUCLEO-L476RG
VCC	3V3
GND	GND
SCL	D15 / PB8 / I2C1_SCL
SDA	D14 / PB9 / I2C1_SDA

I2C1_SDA
I2C1_SCL

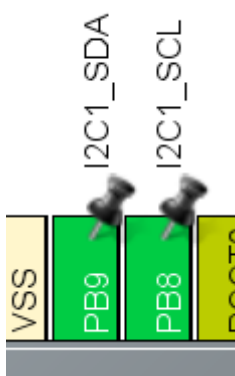


como estava no projeto:

Logo: PB7 é SDA (data)
PB6 SCL (clock)

na placa que estou

Mudei para:

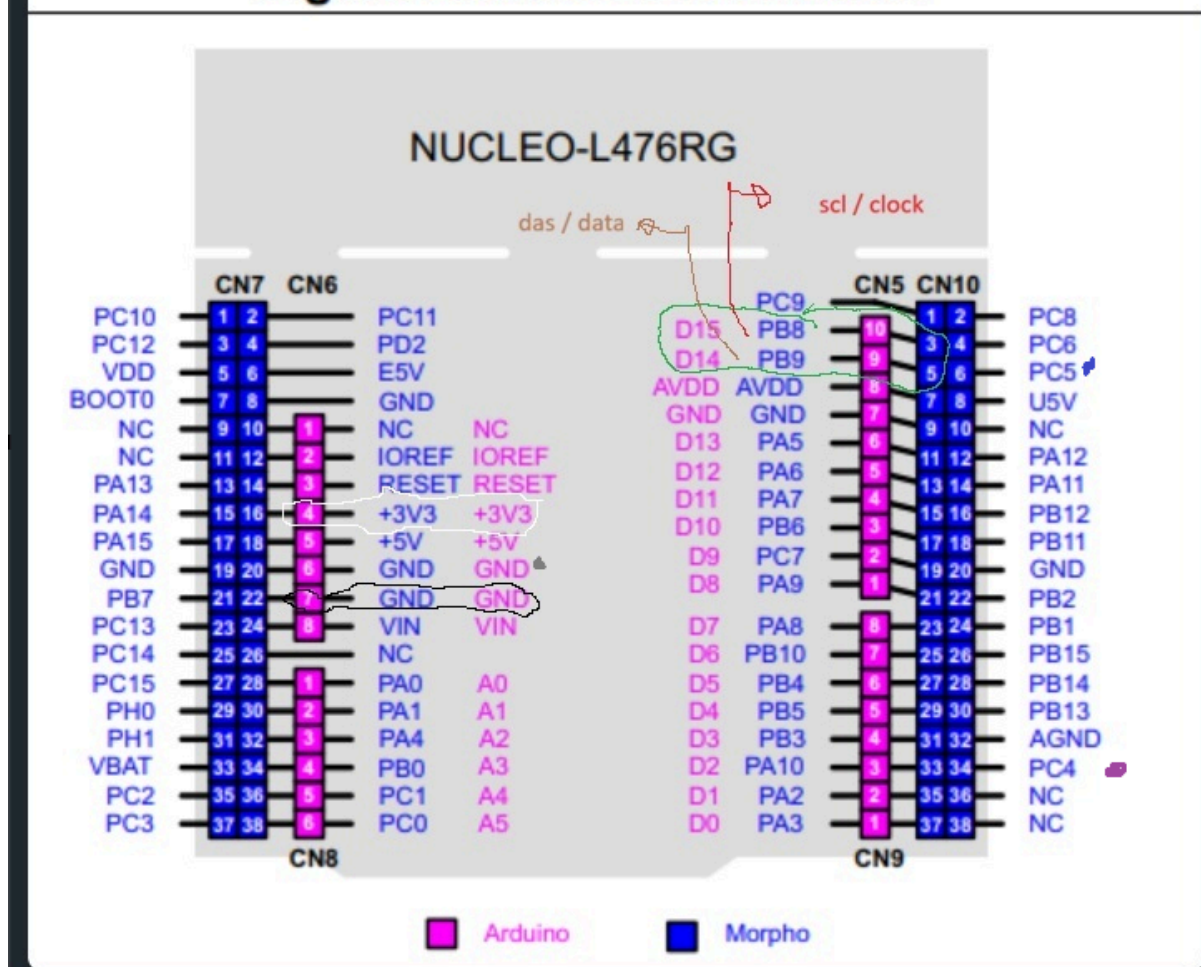


PB8 - Clock

PB9 - Data

PB8 = I2C1_SCL
PB9 = I2C1_SDA

Figure 23. NUCLEO-L476RG

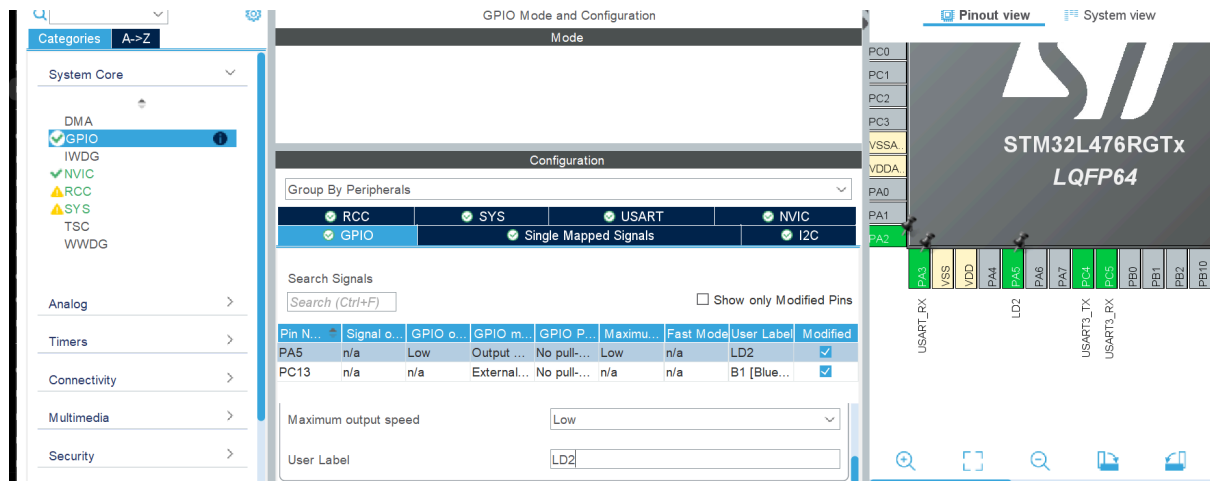
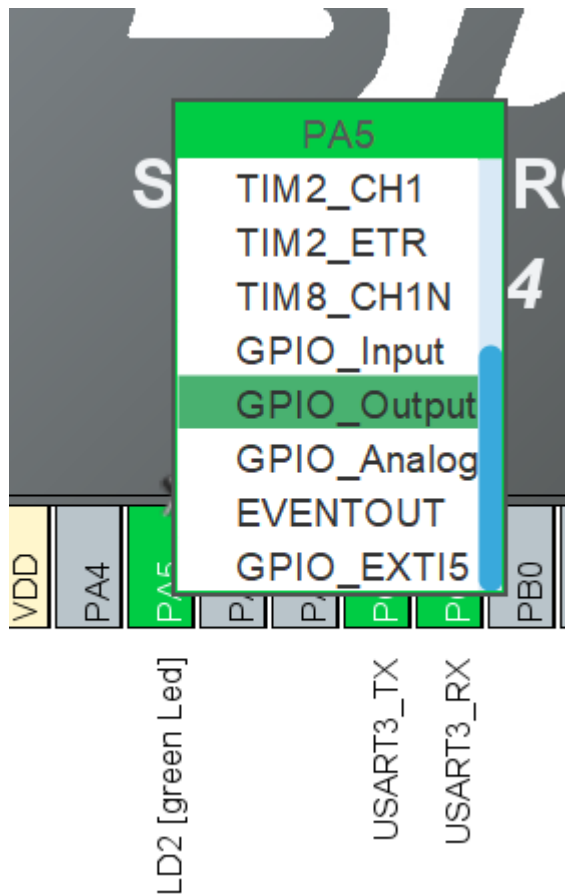


conexões físicas:

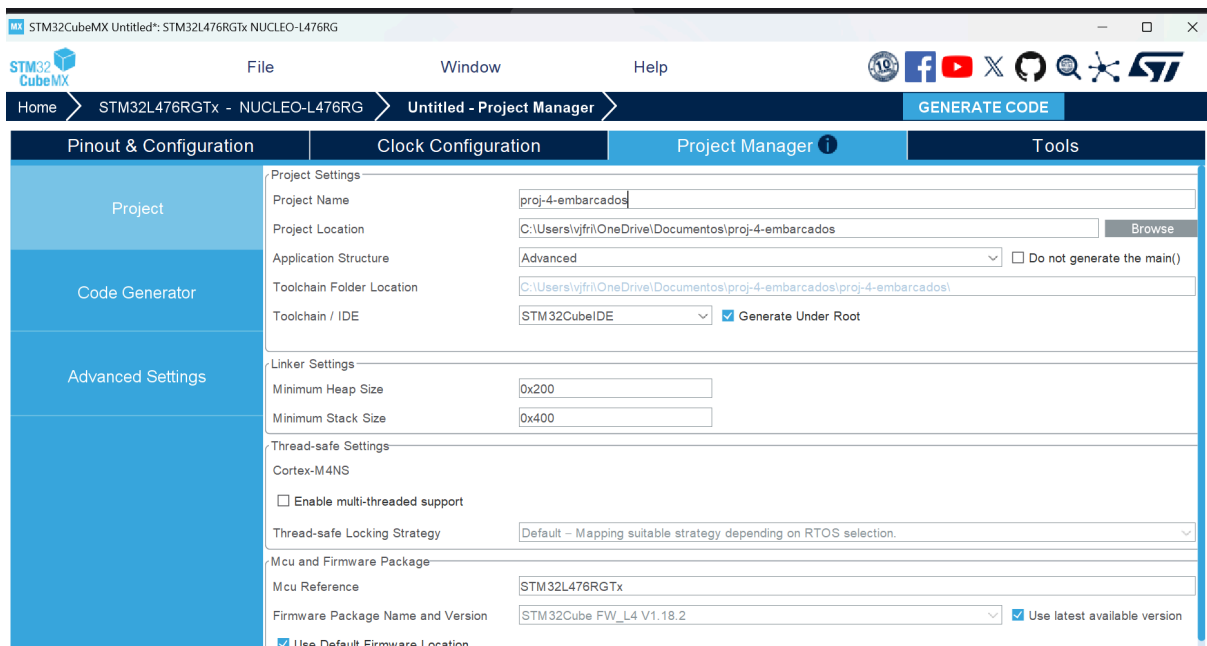
plaquinha x nucleo com MachoxFemea e FemeaxFemea

VCC	---	Jumper branco MxF	---	3v3
GND	---	Jumper preto MxF	---	GND
DATA	---	Jumper Marrom FxF	---	PB9
CLOCK	---	Jumper vermelho FxF	---	PB8

led para debug



por fim,



selecionar IDE correta!

tip: habilitar o float, pra pegar as exibições

Código:

Os trechos foram:

1. includes

```
/* USER CODE BEGIN Includes */
#include <string.h>
#include <stdio.h>
#include <stdlib.h>
/* USER CODE END Includes */
```

2. Variáveis globais

```

/* USER CODE BEGIN PV */

/*
 * Endereco I2C do modulo PCF8591.
 * O modulo usa endereco 7 bits = 0x48.
 * A HAL pede o endereco deslocado 1 bit para a esquerda.
 */
#define PCF8591_ADDR      (0x48 << 1)

/*
 * Bit 6 = 1 habilita a saida analogica AOUT/DAC.
 * Bits 1 e 0 escolhem o canal ADC:
 * 00 = AIN0
 * 01 = AIN1
 * 10 = AIN2
 * 11 = AIN3
 */
#define PCF8591_CTRL_DAC_ENABLE  0x40

#define UART_RX_BUFFER_SIZE      64

uint8_t uart_rx_byte;
char uart_rx_buffer[UART_RX_BUFFER_SIZE];
uint8_t uart_rx_index = 0;

uint8_t i2c_tx_buffer[2];
uint8_t i2c_rx_buffer[2];

volatile uint8_t esperando_leitura_adc = 0;
volatile uint8_t i2c_ocupado = 0;

uint8_t canal_atual = 0;
uint8_t dac_atual = 0;

/* USER CODE END PV */

```

3. Protótipos de função / chamadas

```

/* USER CODE BEGIN PFP */

void UART_Print(char *msg);
void Processar_Comando(char *cmd);
void PCF8591_Ler_Canal_IT(uint8_t canal);
void PCF8591_Set_DAC_IT(uint8_t valor);

/* USER CODE END PFP */

```

4. funções

```

/* USER CODE BEGIN 0 */

void UART_Print(char *msg)
{
    HAL_UART_Transmit(&huart3, (uint8_t *)msg, strlen(msg), 100);
}

```

```

void PCF8591_Ler_Canal_IT(uint8_t canal)
{
    if (i2c_ocupado)
    {
        UART_Print("[ERRO] I2C ocupado. Tente novamente.\r\n");
        return;
    }

    if (canal > 3)
    {
        UART_Print("[ERRO] Canal invalido.\r\n");
        return;
    }

    canal_atual = canal;
    esperando_leitura_adc = 1;
    i2c_ocupado = 1;

    /*
     * Controle do PCF8591:
     * 0x40 habilita DAC/AOUT.
     * canal escolhe AIN0, AIN1, AIN2 ou AIN3.
     */
    i2c_tx_buffer[0] = PCF8591_CTRL_DAC_ENABLE | canal;

    HAL_GPIO_TogglePin(LD2_GPIO_Port, LD2_Pin);

    /*
     * Primeiro enviamos o byte de controle escolhendo o canal.
     * Quando terminar, o callback HAL_I2C_MasterTxCpltCallback()
     * vai iniciar a leitura.
     */
    if (HAL_I2C_Master_Transmit_IT(&hi2c1, PCF8591_ADDR, i2c_tx_buffer, 1) !=
    HAL_OK)
    {
        UART_Print("[ERRO] Falha ao iniciar transmissao I2C.\r\n");
        esperando_leitura_adc = 0;
        i2c_ocupado = 0;
    }
}

void PCF8591_Set_DAC_IT(uint8_t valor)
{
    if (i2c_ocupado)
    {
        UART_Print("[ERRO] I2C ocupado. Tente novamente.\r\n");
        return;
    }

    dac_atual = valor;
    esperando_leitura_adc = 0;
    i2c_ocupado = 1;

    /*

```

```

* Para ajustar o DAC:
* Byte 1 = controle com DAC habilitado
* Byte 2 = valor de 0 a 255
*/
i2c_tx_buffer[0] = PCF8591_CTRL_DAC_ENABLE;
i2c_tx_buffer[1] = dac_atual;

HAL_GPIO_TogglePin(LD2_GPIO_Port, LD2_Pin);

if (HAL_I2C_Master_Transmit_IT(&hi2c1, PCF8591_ADDR, i2c_tx_buffer, 2) !=
HAL_OK)
{
    UART_Print("[ERRO] Falha ao iniciar envio ao DAC.\r\n");
    i2c_ocupado = 0;
}
}

void Processar_Comando(char *cmd)
{
    char msg[100];

    if (strcmp(cmd, "Read_AIN0") == 0)
    {
        UART_Print("[CMD] Read_AIN0 recebido.\r\n");
        PCF8591_Ler_Canal_IT(0);
    }
    else if (strcmp(cmd, "Read_AIN1") == 0)
    {
        UART_Print("[CMD] Read_AIN1 recebido.\r\n");
        PCF8591_Ler_Canal_IT(1);
    }
    else if (strcmp(cmd, "Read_AIN3") == 0)
    {
        UART_Print("[CMD] Read_AIN3 recebido.\r\n");
        PCF8591_Ler_Canal_IT(3);
    }
    else if (strncmp(cmd, "Set_DAC_", 8) == 0)
    {
        int valor = atoi(&cmd[8]);

        if (valor < 0 || valor > 255)
        {
            UART_Print("[ERRO] Valor do DAC deve estar entre 0 e 255.\r\n");
            return;
        }

        sprintf(msg, "[CMD] Set_DAC_%d recebido.\r\n", valor);
        UART_Print(msg);

        PCF8591_Set_DAC_IT((uint8_t)valor);
    }
    else
    {
        UART_Print("[ERRO] Comando invalido.\r\n");
    }
}

```

```

    UART_Print("Use: Read_AIN0, Read_AIN1, Read_AIN3 ou Set_DAC_120\r\n");
  }
}

/* USER CODE END 0 */

```

5. main

```

/* USER CODE BEGIN 2 */

HAL_GPIO_WritePin(LD2_GPIO_Port, LD2_Pin, GPIO_PIN_RESET);

UART_Print("\r\n===== \r\n");
UART_Print(" Projeto 4 - I2C com PCF8591\r\n");
UART_Print(" NUCLEO-L476RG + USART3 + I2C1\r\n");
UART_Print("===== \r\n");
UART_Print("Comandos disponiveis:\r\n");
UART_Print("Read_AIN0\r\n");
UART_Print("Read_AIN1\r\n");
UART_Print("Read_AIN3\r\n");
UART_Print("Set_DAC_0 ate Set_DAC_255\r\n\r\n");

/*
 * Comeca a receber caracteres da USART3 por interrupcao.
 * Cada caractere digitado no terminal chega no callback.
 */
HAL_UART_Receive_IT(&huart3, &uart_rx_byte, 1);

/* USER CODE END 2 */

```

6. callback função de retorno / chamada da resposta

```

/* USER CODE BEGIN 4 */

void HAL_UART_RxCpltCallback(UART_HandleTypeDef *huart)
{
  if (huart->Instance == USART3)
  {
    /*
     * Se recebeu ENTER, finaliza o comando.
     * O terminal pode mandar '\r', '\n' ou os dois.
     */
    if (uart_rx_byte == '\r' || uart_rx_byte == '\n')
    {
      if (uart_rx_index > 0)

```

```

    {
        uart_rx_buffer[uart_rx_index] = '\0';

        UART_Print("\r\n[UART3 RX] Comando recebido: ");
        UART_Print(uart_rx_buffer);
        UART_Print("\r\n");

        Processar_Comando(uart_rx_buffer);

        uart_rx_index = 0;
        memset(uart_rx_buffer, 0, UART_RX_BUFFER_SIZE);
    }
}
else
{
    /*
     * Guarda caractere recebido no buffer.
     */
    if (uart_rx_index < UART_RX_BUFFER_SIZE - 1)
    {
        uart_rx_buffer[uart_rx_index] = uart_rx_byte;
        uart_rx_index++;
    }
    else
    {
        uart_rx_index = 0;
        memset(uart_rx_buffer, 0, UART_RX_BUFFER_SIZE);
        UART_Print("\r\n[ERRO] Comando muito grande.\r\n");
    }
}

/*
 * Rearma a recepcao por interrupcao.
 * Sem isso, receberia apenas 1 caractere.
 */
HAL_UART_Receive_IT(&huart3, &uart_rx_byte, 1);
}
}

void HAL_I2C_MasterTxCpltCallback(I2C_HandleTypeDef *hi2c)
{
    if (hi2c->Instance == I2C1)
    {
        /*
         * Se a transmissao foi para selecionar canal ADC,
         * agora iniciamos a leitura.
         */
        if (esperando_leitura_adc)
        {
            /*
             * No PCF8591, a primeira leitura costuma ser dummy/anterior.
             * Por isso lemos 2 bytes e usamos o segundo.
             */
            if (HAL_I2C_Master_Receive_IT(&hi2c1, PCF8591_ADDR, i2c_rx_buffer, 2) !=

```

```

HAL_OK)
{
    UART_Print("[ERRO] Falha ao iniciar leitura I2C.\r\n");
    esperando_leitura_adc = 0;
    i2c_ocupado = 0;
}
}
else
{
    /*
     * Se nao estava esperando ADC, entao era escrita no DAC.
     */
    char msg[80];
    float tensao = (dac_atual * 3.3f) / 255.0f;

    sprintf(msg, "Valor do DAC: %u | %.2f V\r\n", dac_atual, tensao);
    UART_Print(msg);

    i2c_ocupado = 0;

    HAL_GPIO_TogglePin(LD2_GPIO_Port, LD2_Pin);
}
}
}

void HAL_I2C_MasterRxCpltCallback(I2C_HandleTypeDef *hi2c)
{
    if (hi2c->Instance == I2C1)
    {
        char msg[100];

        /*
         * i2c_rx_buffer[0] = valor anterior/dummy
         * i2c_rx_buffer[1] = valor mais confiavel do canal atual
         */
        uint8_t valor_adc = i2c_rx_buffer[1];
        float tensao = (valor_adc * 3.3f) / 255.0f;

        sprintf(msg, "AIN%u: %u | %.2f V\r\n", canal_atual, valor_adc, tensao);
        UART_Print(msg);

        esperando_leitura_adc = 0;
        i2c_ocupado = 0;

        HAL_GPIO_TogglePin(LD2_GPIO_Port, LD2_Pin);
    }
}

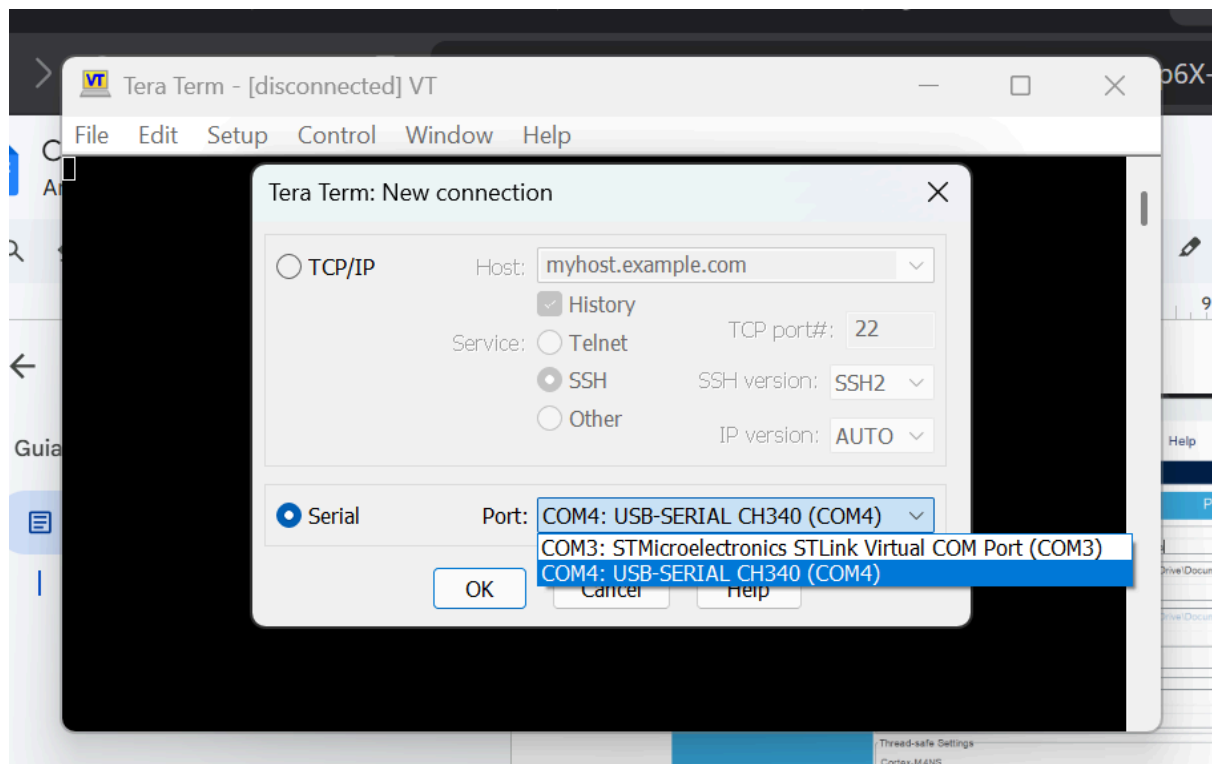
void HAL_I2C_ErrorCallback(I2C_HandleTypeDef *hi2c)
{
    if (hi2c->Instance == I2C1)
    {
        UART_Print("[ERRO] Erro na comunicacao I2C com o PCF8591.\r\n");
    }
}

```



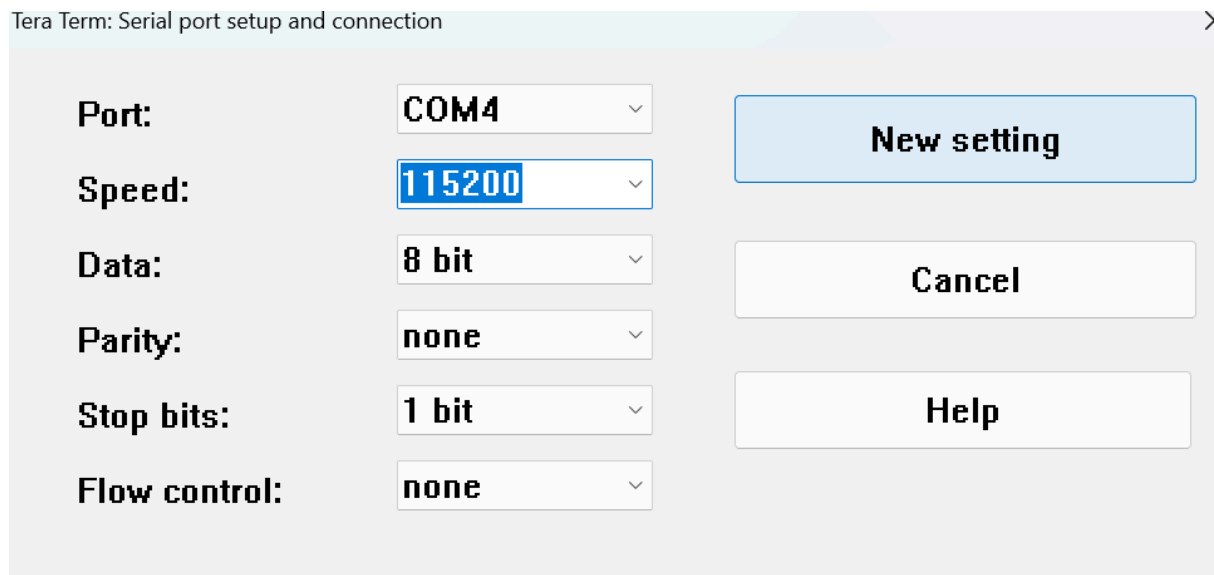
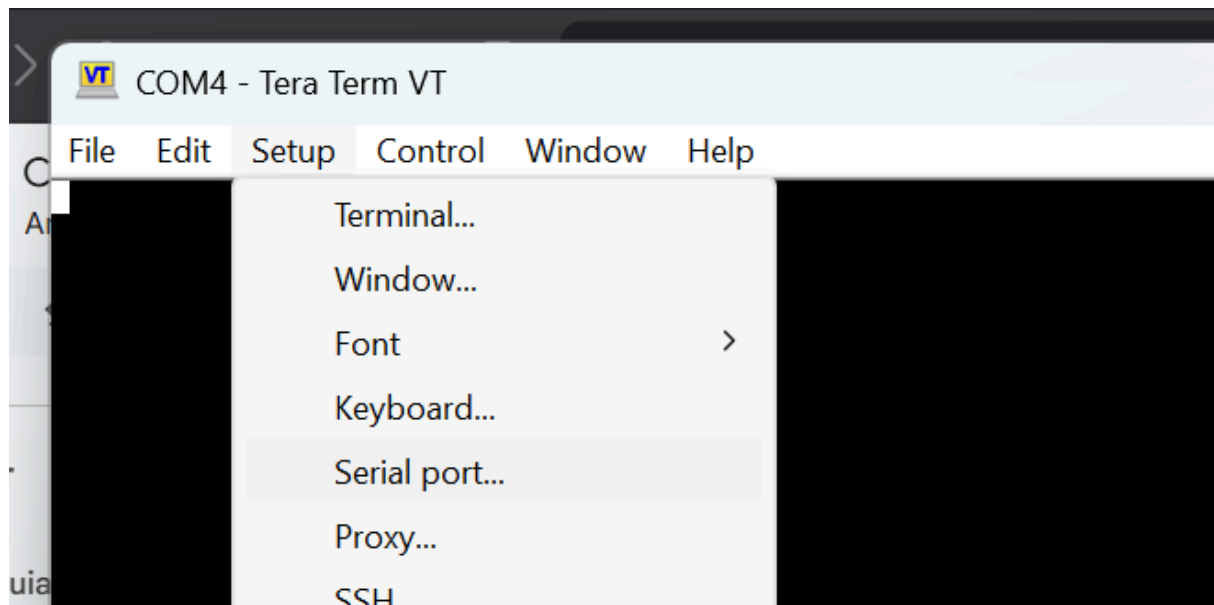
```
esperando_leitura_adc = 0;  
i2c_ocupado = 0;  
  
HAL_GPIO_WritePin(LD2_GPIO_Port, LD2_Pin, GPIO_PIN_RESET);  
}  
}  
  
/* USER CODE END 4 */
```

Configuração do terminal:



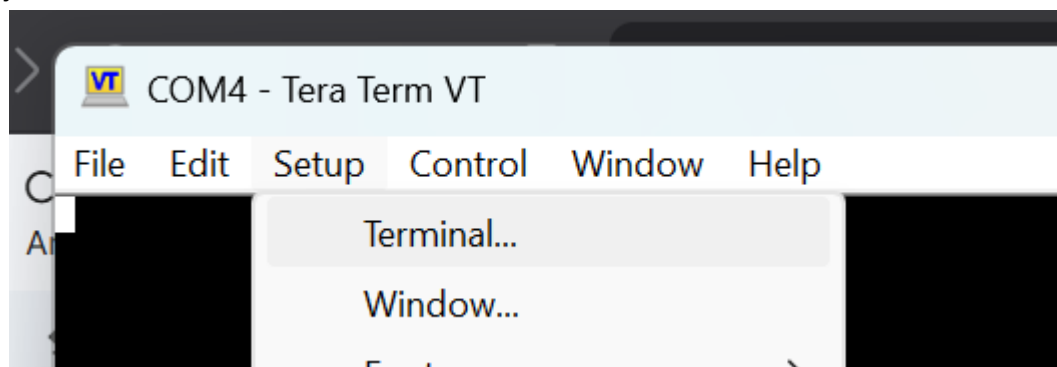
COM 3: USB da placa

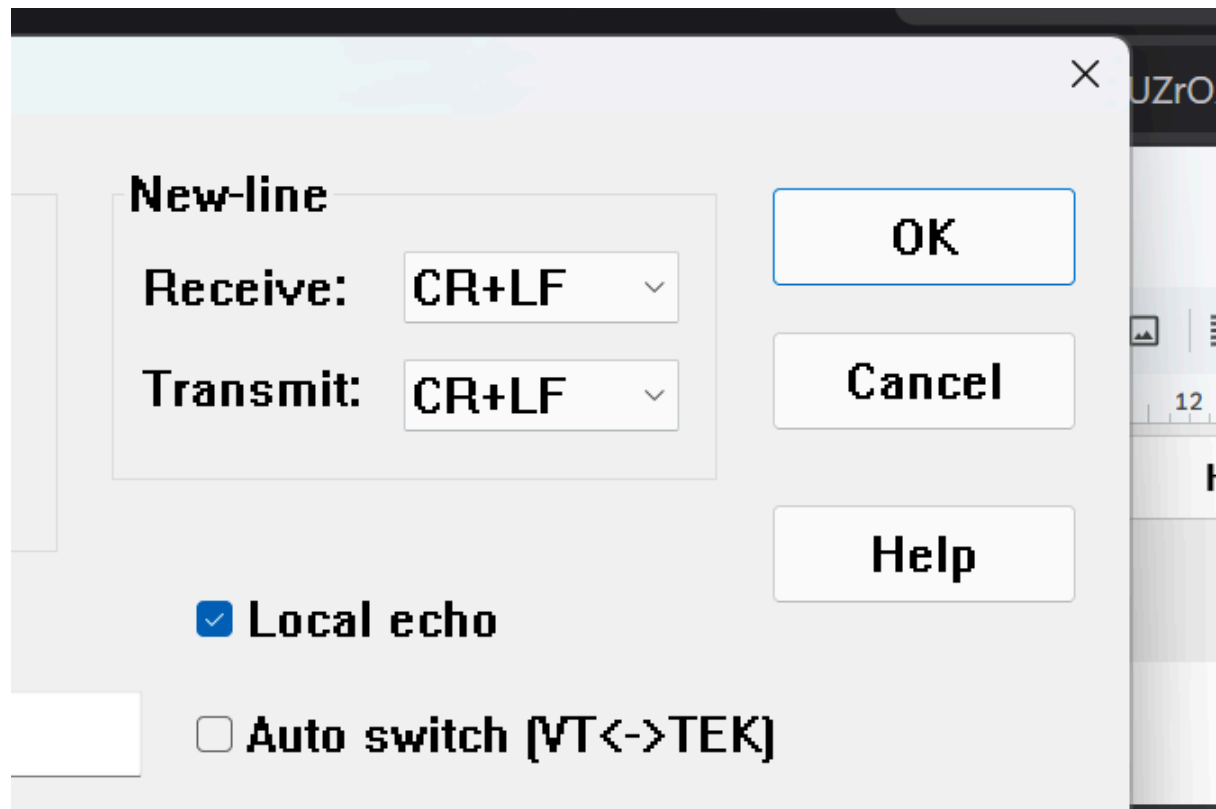
COM 4: adaptador



ajusta o baud rate

já em "terminal":





permitir I/O e visualização do digitado

—

Resultados:

```
[UART3 RX] Comando recebido: Read_AIN0
[CMD] Read_AIN0 recebido.
AIN0: 239 | V
Read_AIN0

[UART3 RX] Comando recebido: Read_AIN0
[CMD] Read_AIN0 recebido.
AIN0: 111 | V
```

```
[UART3 RX] Comando recebido: Read_AIN0  
[CMD] Read_AIN0 recebido.  
AIN0: 43 | V
```

Leituras:

239/255 (luz apagada)

111/255 (luz acesa)

43/255 (luz acesa + lanterna do celular)

```
Read_AIN1  
  
[UART3 RX] Comando recebido: Read_AIN1  
[CMD] Read_AIN1 recebido.  
AIN1: 228 | V  
Read_AIN1  
  
[UART3 RX] Comando recebido: Read_AIN1  
[CMD] Read_AIN1 recebido.  
AIN1: 224 | V  
Read_AIN1  
  
[UART3 RX] Comando recebido: Read_AIN1  
[CMD] Read_AIN1 recebido.  
AIN1: 229 | V  
█
```

224 - temperatura ambiente

228 - soprando

229 - ventilador

```
[UART3 RX] Comando recebido: Read_AIN3
```

```
[CMD] Read_AIN3 recebido.
```

```
AIN3: 133 | V
```

```
Read_AIN3
```

```
[UART3 RX] Comando recebido: Read_AIN3
```

```
[CMD] Read_AIN3 recebido.
```

```
AIN3: 165 | V
```

```
Read_AIN3
```

```
[UART3 RX] Comando recebido: Read_AIN3
```

```
[CMD] Read_AIN3 recebido.
```

```
AIN3: 113 | V
```

133 - disposição padrão

165 - leve giro anti-horario

133 Leve giro horário

por tensão maxima:

```
=====
Projeto 4 - I2C com PCF8591
NUCLEO-L476RG + USART3 + I2C1
=====
```

Comandos disponiveis:

Read_AIN0

Read_AIN1

Read_AIN3

Set_DAC_0 ate Set_DAC_255

Set_DAC_255

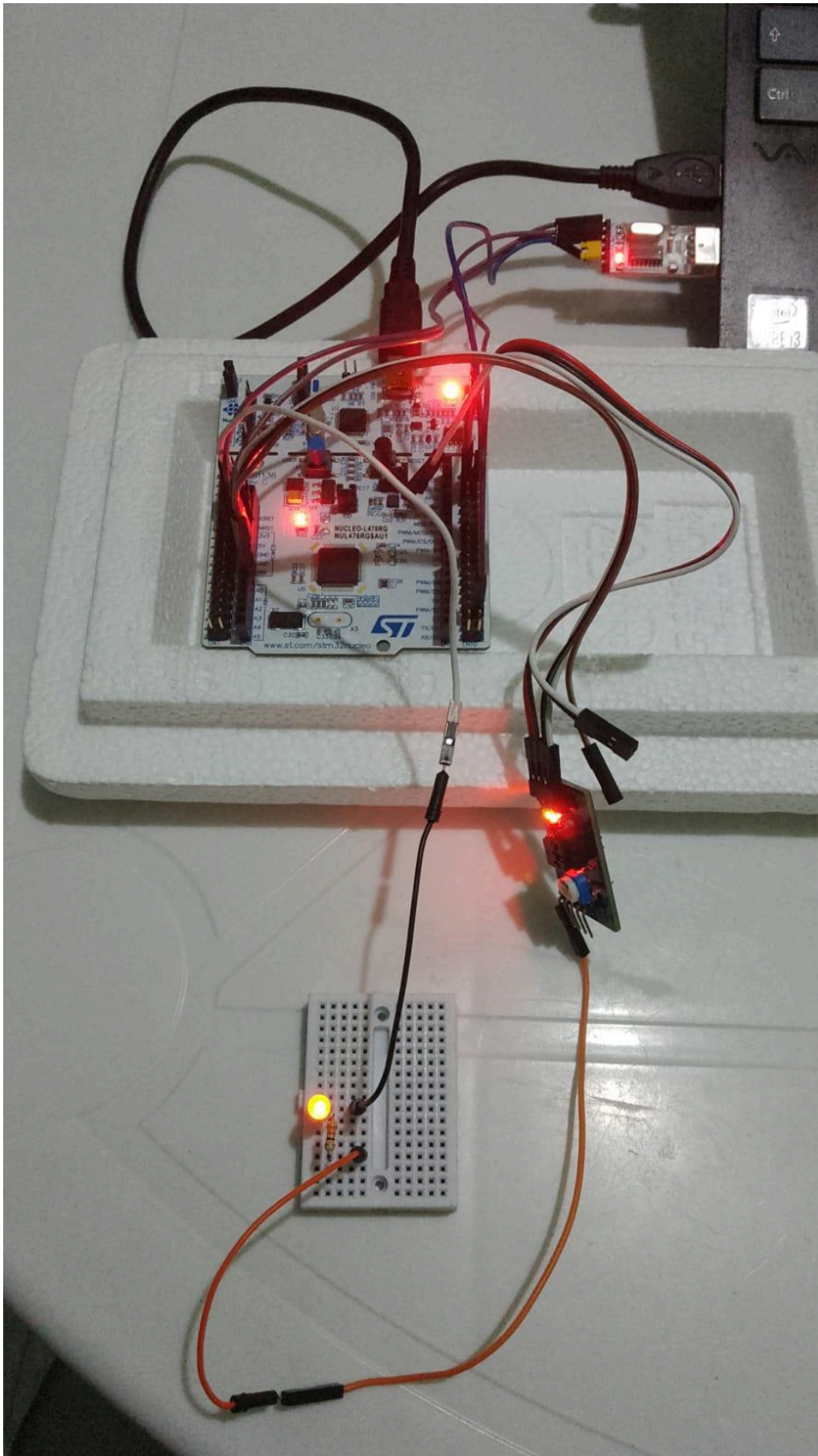
[UART3 RX] Comando recebido: Set_DAC_255

[CMD] Set_DAC_255 recebido.

Valor do DAC: 255 | 3300mV





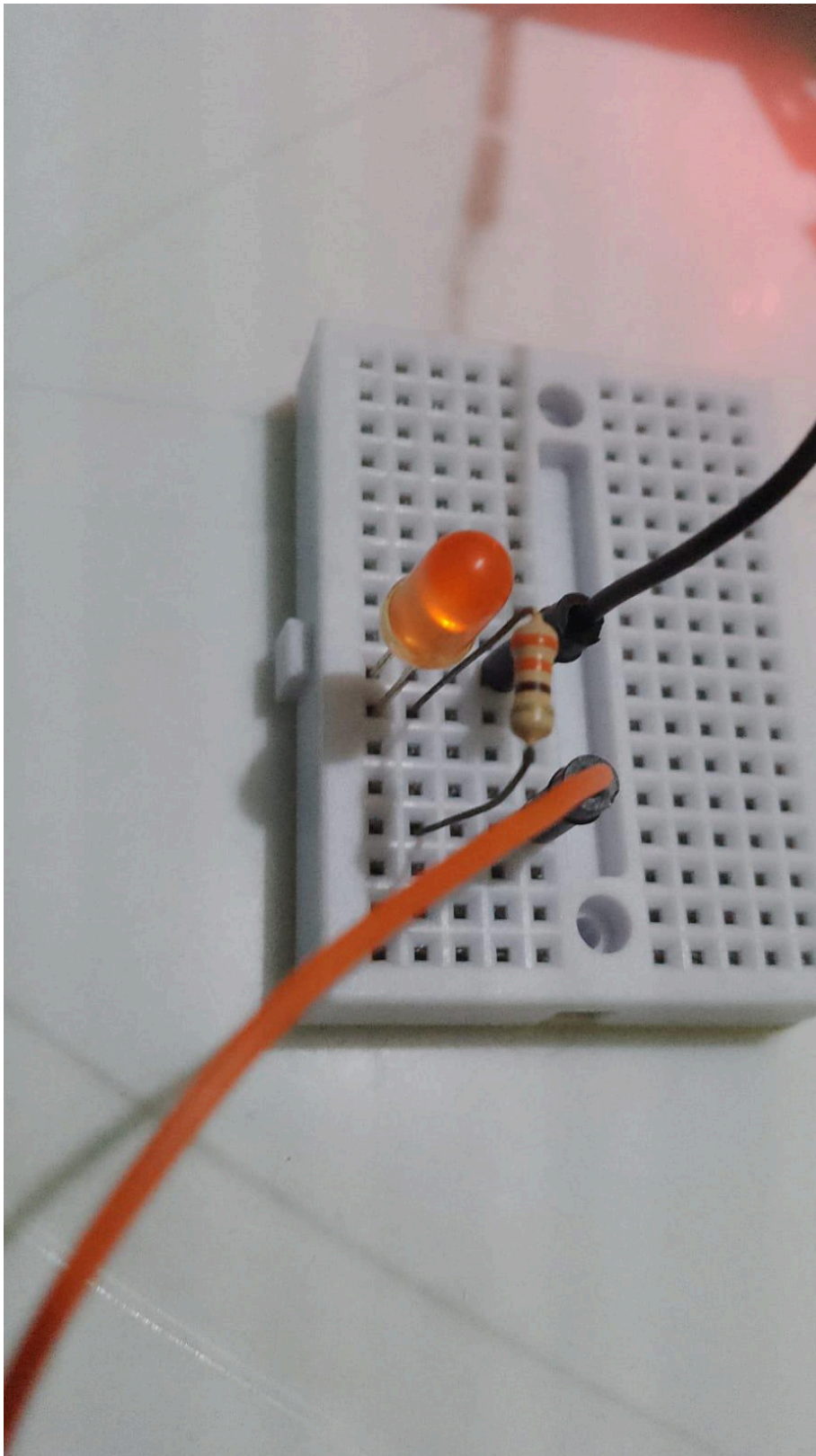


Led acende

tensão média:

```
[UART3 RX] Comando recebido: Set_DAC_128  
[CMD] Set_DAC_128 recebido.  
Valor do DAC: 128 | 1656mV  
Set_DAC_200
```





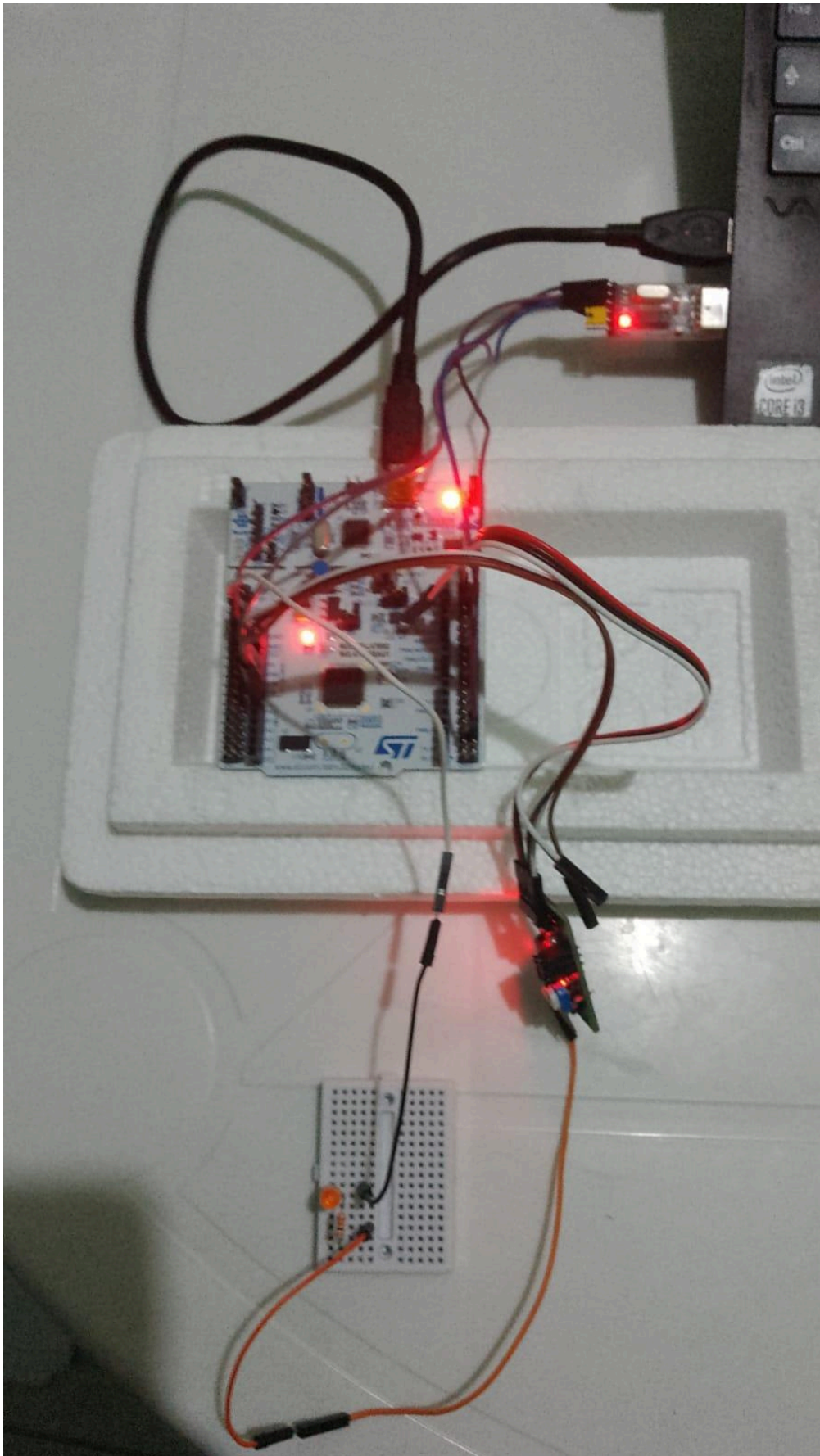
led levemente aceso

para zero tensão:

[UART3 RX] Comando recebido: Set_DAC_0

[CMD] Set_DAC_0 recebido.

Valor do DAC: 0 | 0mV



led apagado.

